PROJET GPGPU · GISTRE_3

Séparation fond / objets mobiles

Un filtre GStreamer en CUDA, de 1,7 à 68 FPS



Valentin Delhommeau · Jon Gouspy · Ilan Mayeux · Alexandre Lacroix

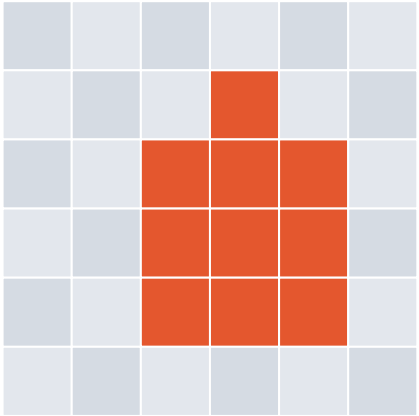
Sommaire

01 Algorithme

02 Protocole de mesure

03 Optimisations GPU, de v1.1.0 à v1.1.8

04 Synthèse



L'algorithme en cinq étapes

1

Estimation du fond

K couleurs pondérées par pixel ; le fond est la plus fréquente

2

Masque

Écart de couleur entre la frame et le fond

3

Ouverture

Érosion puis dilatation : retire le bruit

4

Hystérésis

Deux seuils : on garde ce qui touche un pixel fort

5

Rendu

Les pixels en mouvement passent en rouge

Protocole de mesure

Machine

GPU

RTX 5050 Laptop, 8 Go

CPU

Intel i5-13500HX

CUDA

13.2

Profileur

Nsight Systems 2025.6

Vidéo de test	Résolution
3630-172488409_large.mp4	1920 × 1080
nuits_blanches.mp4	1920 × 1080
ACET.mp4	776 × 1380

Ce que l'on mesure

- Temps de chaque kernel et nombre de lancements
- Allocations, copies et synchronisations
- FPS = frames / durée totale du traitement

Feuille de route des optimisations



Point de départ : CPU contre GPU naïf

v1.1.0

1,7 FPS

Version CPU mono-thread

33 FPS

Portage GPU naïf (v1.1.0) : ×20

Mais le portage naïf garde cinq défauts

- **Allocations à chaque frame**
- **Transferts en double**
- **Masque en float**
- **Ouverture : 81 lectures par pixel**
- **Hystérésis : 1 pixel par lancement**

Réduire la mémoire et les transferts

v1.1.1–2

Avant

- Masque en float (4 octets)
- Buffers alloués à chaque frame
- Fond renvoyé au CPU puis recopié

Après

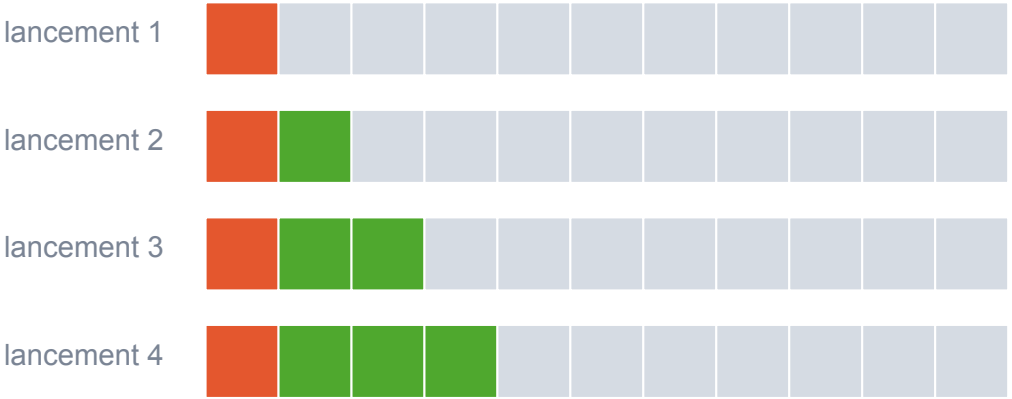
- Masque en uint8_t (1 octet)
- Buffers alloués une seule fois
- Frame envoyée une fois, fond gardé sur le GPU

-10 % de temps GPU seulement : l'hystérésis coûtait 10 fois plus que les copies.

Hystérésis : le goulot principal

v1.1.3

Un seul pixel de propagation par lancement



172

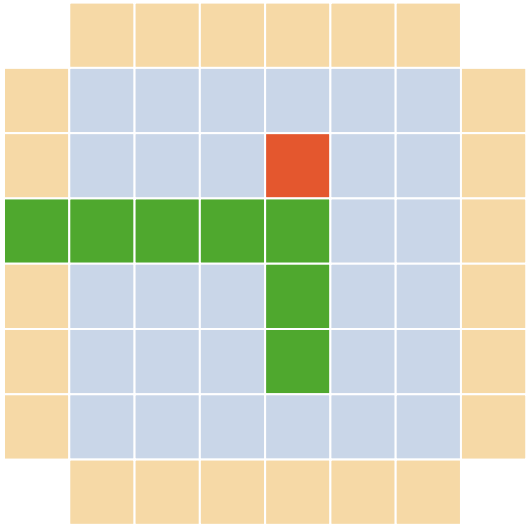
lancements par frame

≈ 84 % du temps GPU

Un objet de N pixels demande N lancements, chacun suivi d'une synchronisation avec le CPU.

Hystérésis par tuiles en mémoire partagée

v1.1.3



Halo : bords des tuiles voisines

Pixel fort

Pixel marqué

1 · Chargement

La tuile et son halo passent en mémoire partagée.

2 · Propagation locale

Le bloc propage jusqu'à ce que plus rien ne change dans la tuile.

3 · Écriture

Seuls les nouveaux pixels marqués repartent en mémoire globale.

Un lancement propage sur toute une tuile au lieu d'un seul pixel.

Hystérésis par tuiles : résultats

v1.1.3

172 → 8

lancements par frame

36 → 66 FPS

de bout en bout : le plus grand gain du projet

Chaque lancement est un peu plus long, mais il en faut 22 fois moins.

Accès mémoire par le pitch

v1.1.4

Tous les kernels indexent maintenant les images via le pitch.

Une ligne du masque en 1080p :



1 920 octets = 15 × 128 octets : les lignes étaient déjà alignées

Pas de gain

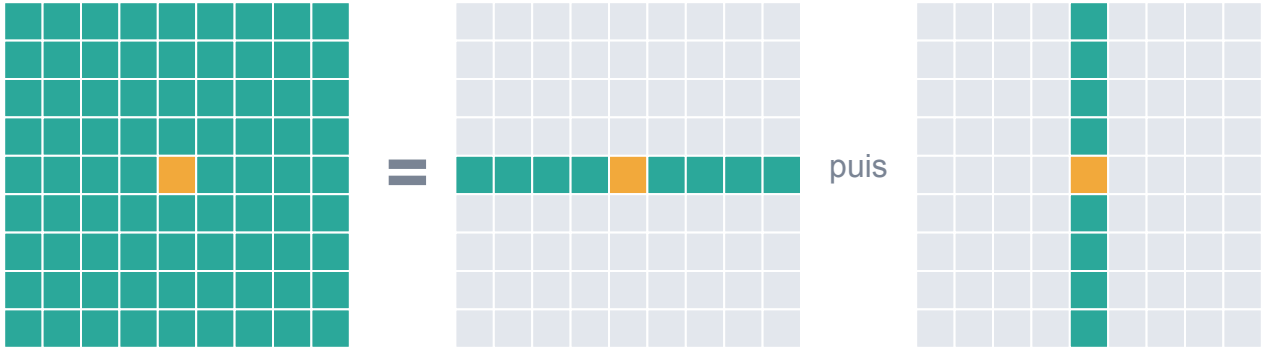
à cette résolution

Conservée quand même

Code cohérent et robuste pour les largeurs non multiples de 128 octets.

Ouverture séparable

v1.1.5



Carré : 81 lectures

Ligne puis colonne : 18 lectures

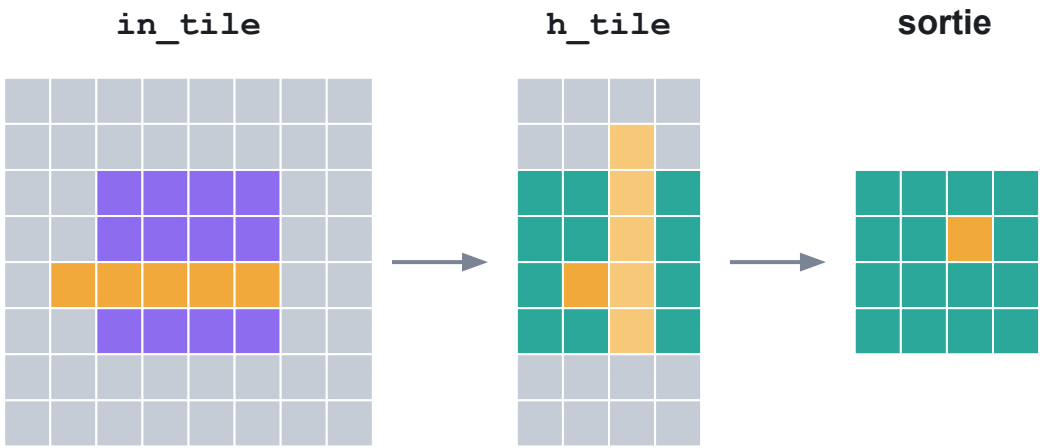
÷1,8

temps de l'ouverture

Décomposé en une passe sur la ligne, puis sur la colonne.

Ouverture en mémoire partagée

v1.1.6



Une seule lecture et une seule écriture en mémoire globale par opération : tout le reste se fait en mémoire partagée.

4 → 2

lancements par frame

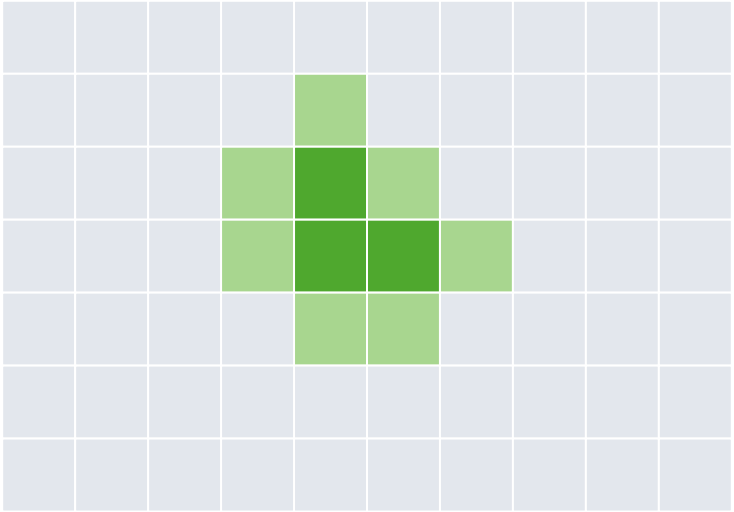
-32 %

temps de l'ouverture

Hystérésis : sauter les blocs inactifs

v1.1.7

Seuls les voisins d'un changement sont relancés



- Modifié au lancement précédent
- Voisin : relancé
- Inchangé : sort tout de suite

÷2

durée d'un lancement d'hystérésis

Même nombre de lancements, beaucoup moins de travail par lancement

Estimation du fond : de l'AoS au SoA

v1.1.8

Avant (AoS) : les états de chaque pixel sont groupés



Après (SoA) : un tableau par champ

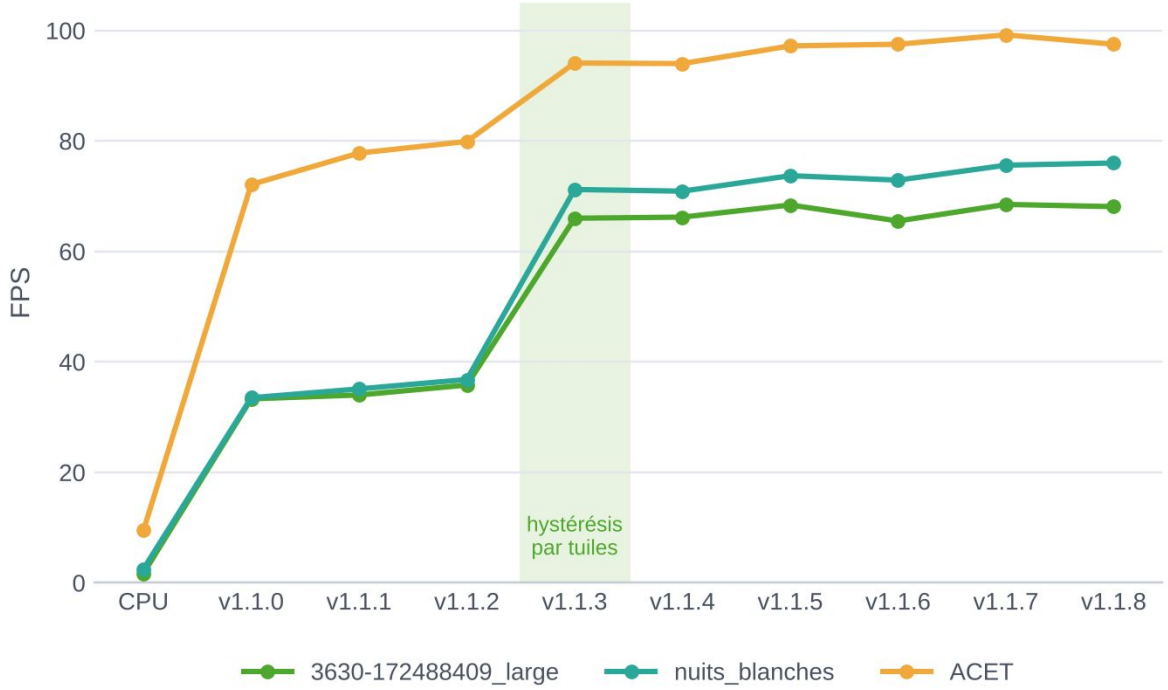


Les threads voisins lisent des adresses contiguës, sans changer l'algorithme.

÷1,9

temps d'un appel de l'estimation du fond

Synthèse : FPS par version



Accélération finale par rapport au CPU

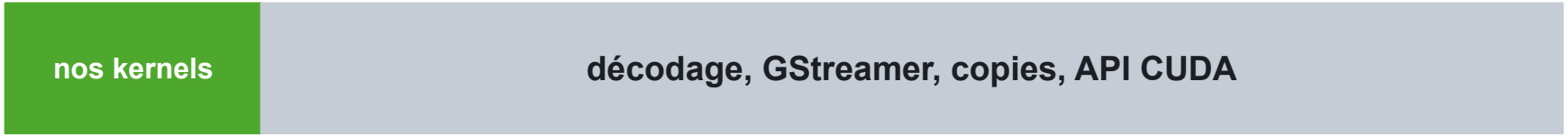
×41
3630 (1080p)

×30
nuits_blanches (1080p)

×10
ACET (776 × 1380)

Où passe le temps aujourd'hui ?

Une frame en 1080p



≈ 16 %

du temps d'une frame passe dans nos kernels

Conséquence

Les FPS plafonnent : accélérer encore les kernels ne change presque plus rien de bout en bout.

Merci !
Des questions ?

Groupe GISTRE_3

